Problem 1: (3 points)

Use Percolation

Consider an epidemic that spreads over a directed network according to the `SIR` model, with infections following the direction of the edges. Consider the set of nodes that are *infectious* at the beginning of the epidemic (i.e., at $t = 0$) — we refer to them as the **seed set** of the epidemic. For a given seed set S, let $f(S)$ be its *expected spread*, i.e., the expected number of nodes (including the seed set) that become eventually infected.

Let X and Y be two seed sets, such that $X \subseteq Y$. Prove or disprove the following:

$$f(X) \leq f(Y) \tag{1}$$

In other words, if additional nodes are infectious initially, do we expect the virus to spread to more nodes eventually?

Problem 2: (7 points)

Consider a network with graph structure given by an *undirected* graph $G(V, E)$. Consider also a virus that is possible to spread over the network in either direction of the edges, according to the SIR model with transmission probability p.

For this problem, while any node in the network may become infected by the virus eventually, only a given *candidate set* $C \subseteq V$ of nodes may be part of the *seed set* – or, in other words, only nodes from C may be infectious at time $t = 0$.

Our task is to identify a seed set $S \subseteq C$ of k nodes ($k = \|S\| < \|C\|$) that has a large expected spread. Towards this end, we employ the following **greedy** algorithm to build a seed set iteratively: the algorithm starts with an empty seed set; in each iteration, it adds to the seed set the next single node $u^* \in C$ which would maximize the expected spread; and stops after k iterations (see the pseudocode in Algorithm 1).

Algorithm 1 *Greedy* (G, C, k)

Input: Graph $G(V, E)$; Candidate nodeset $C \subseteq V$; Positive integer $k < \|C\|$

```
S := ∅                                          ▷ initialize to empty set
for i = 1 .. k do
    u* := arg max_{u∈C} f(S ∪ {u})              ▷ find additional node to maximize expected thread
    S := S ∪ {u*}                               ▷ update seed and candidate set
    C := C − {u*}
end for
return S
```

Experiments: Run **greedy** on the `h3_graph_data.edgelist` dataset to select a seed set of size $k = 5$ and report your findings. Specifically:

1. Pick randomly a candidate set C of 7 nodes from V.
2. Run the **greedy** algorithm for $k = 5$ using this candidate set C.

f = mean(I + R)

- To (approximately) calculate the expected spread $f(S)$ of a seed set S, produce $\sigma = 5$ simulations of SIR with this seed set and use the average spread you observe as $f(S)$.
- Keep track of $f(S)$ at the end of each iteration of the algorithm, i.e., when the size of S is $1, 2, \ldots, k$.

Perform the experiment above for three randomly-chosen candidate sets C and for two values of infection probability $p = \{0.1, 0.5\}$. This means you will have a total of 6 runs of the **greedy** algorithm.

Results: For each value of p, produce a plot in which the x-axis corresponds to the size of the seed set S in the experiments above (i.e., the x-values should be $1, 2, \ldots, k$) and the y-axis corresponds to $f(S)$. The plot should include one data point for each candidate set and each size of seed set. Use a different color (red, blue, green) for the points of each of the three candidate sets.

Discussion: Discuss your findings from the plots. Compare the eventual spread across different candidate sets and infection probabilities. Do your results suggest an answer to Problem 1?

Running time: Report the number of minutes it takes to run your code in its entirety (e.g., 'less than a minute' or '5 minutes'), and for what computer you obtained it (e.g., 'Apple M1, 4 cores, 8 GBs RAM').

Implementation:

Where necessary to use an implementation of SIR, you can implement it yourself (e.g., in `Networkx`); or use an existing implementation (e.g., Python's `epydemic` library). If possible, ensure the reproducibility of your code using random seeds. This might not be possible if using the `epydemic` library (see link for more details). The boilerplate code from Moodle is **optional** and uses the Python `epydemic` library. If you wish to use it install the library in your python environment.

Instructions:

Submit a single zip file that contains the following:

- `solutions.pdf`: A writeup of your solution for Problem 1; and plots, observations for Problem 2.
- `hw3_p2.py` or `hw3_p2.ipynb`: code-file(s) for running the experiments, creating a log-file, and plotting the results. This should include methods for the greedy algorithm and the SIR simulation.
- `README` : a text/markdown file explaining how a single trial can be executed and how the entire experiment can be rerun.

Keep track metrics/statistics after simulating iterati

- `results.csv`: log-file of results from your experiment as described above.
- `h3_graph_data.edgelist`: original dataset file.